

Task 1

Requirements under test The table maps each assignment task to the automated tests that verify it. All 14 tests were run with `python -m pytest -v` and all passed (output in Appendix D).

Task	Requirement	Verifying tests	Result
A2	An increment applies its delta once; a repeated idempotency key is not applied again (was_duplicate = true); Get of an unknown counter returns found = false; a retry after a timeout is safe	test_increment_applies_delta, test_duplicate_key_not_reapplied, test_get_missing_counter, test_retry_after_timeout_is_safe (tests/test_counter.py)	passed
A3	Concurrent clients neither lose nor duplicate updates	test_concurrent_increments_exact (tests/test_counter.py)	passed
B1	Lamport rules: +1 before a local event, max(own, received) + 1 on receive; logged events follow the required format	test_local_event_increments_clock, test_receive_takes_max_plus_one, test_receive_with_smaller_value_still_increments (tests/test_clocks.py); event trace in logs/b2_trace.txt (Appendix B)	passed
C1	Three replicas; a write commits with 2 of 3 acknowledgements; below the majority the client reports failure; live replicas converge	test_majority_commit_two_acks, test_no_commit_below_majority, test_replicas_converge (tests/test_counter.py); test_replica_crash_mid_request_still_commits (tests/test_failures.py)	passed

Duplicate suppression and timeout-with-retry are additionally verified under failure injection by test_duplicate_request_moves_value_once and test_induced_timeout_retry_moves_exactly_once (tests/test_failures.py, see Section 4).

Task 2

Test environment

Tests were executed with `python -m pytest -v` (configuration in pytest.ini). The tests start and stop their own servers on free ports; there are no manual steps. Replicas and clients ran on one machine (localhost).

Item	Value
OS	Windows 11 Home Single Language, version 25H2

Machine	11th Gen Intel Core i5-1135G7 @ 2.40 GHz, 8 GB RAM (7.8 GB available), Intel Iris Xe Graphics
Python	3.13.2 (virtual environment .venv)
Key packages	grpcio 1.84.0, grpcio-tools 1.84.0, protobuf 7.36.2, pytest 9.1.1, pluggy 1.6.0 (full pip freeze in Appendix A)
Protocol compiler	libprotoc 35.1

Task 3

Test-case table Actual results are the values asserted by each test in the run shown in Appendix D (14 passed in 17.59 s).

ID	Purpose (test name)	Expected result	Actual result	Pass/fail
TC-01	A local event advances the Lamport clock (test_local_event_increments_clock)	Two local events return L=1 and L=2	Returned 1 and 2	pass
TC-02	Receive with a larger timestamp (test_receive_takes_max_plus_one)	After L=1, receiving 10 gives $\max(1, 10) + 1 = 11$	Returned 11	pass
TC-03	Receive with a smaller timestamp (test_receive_with_smaller_value_still_increments)	At L=5, receiving 2 gives $\max(5, 2) + 1 = 6$	Returned 6	pass
TC-04	Increment applies the delta once (test_increment_applies_delta)	new_value = 7, was_duplicate = false, Get returns 7	new_value 7, was_duplicate false, Get 7	pass
TC-05	A repeated key is not applied again (test_duplicate_key_not_reapplied)	Second call with the same key: new_value = 5, was_duplicate = true; Get = 5, not 10	new_value 5, was_duplicate true, Get 5	pass
TC-06	Get of an unknown counter (test_get_missing_counter)	found = false, no exception	found false, no exception	pass
TC-07	Concurrent increments (test_concurrent_increments_exact)	2 clients x 1,000 increments give exactly 2,000, no errors	Value 2,000, no errors	pass
TC-08	Retry after a timeout is safe (test_retry_after_timeout_is_safe)	Server delay 300 ms, deadline 100 ms: first call fails with DEADLINE_EXCEEDED; retry with the same key gives was_duplicate = true, value 1	DEADLINE_EXCEEDED, then was_duplicate true, value 1	pass

TC-09	Commit with 2 of 3 acknowledgements (test_majority_commit_two_acks)	One replica stopped: acks = 2 of 3, new_value = 1	acks 2/3, new_value 1	pass
TC-10	No commit below the majority (test_no_commit_below_majority)	Two replicas stopped: the client raises QuorumError; the surviving replica holds x = 1	QuorumError raised, surviving replica x = 1	pass
TC-11	Live replicas converge (test_replicas_converge)	10 writes with 3 replicas, one replica stopped, 5 more writes: both live replicas have a = 8, b = 7	Both replicas a = 8, b = 7	pass
TC-12	Replica crash mid-request (test_replica_crash_mid_request_still_commits)	Replica crash-C dies on its first request: commit with 2 of 3 acks, no exception, crashed process exits with code 1, survivors x = 1	acks 2/3, no exception, exit code 1, survivors x = 1	pass
TC-13	Request duplication (test_duplicate_request_moves_value_once)	Same key sent twice through the quorum client: value 5, not 10; a direct resend to each replica returns was_duplicate = true	Value 5; was_duplicate true on all three replicas	pass
TC-14	Induced timeout with retry (test_induced_timeout_retry_moves_value_exactly_once)	First request delayed 1.5 s, deadline 0.5 s: retry succeeds with new_value = 1; the delayed original is later a duplicate; Get = 1	Retry new_value 1, was_duplicate false; Get 1	pass

Result: 14 of 14 tests passed (Python 3.13.2, pytest 9.1.1, Windows 11).

Task 4

Failure-injection results

Each scenario is an automated test that asserts an externally observable invariant. Replicas run as separate processes (tests/test_failures.py); the log excerpts come from logs/failures/ (full excerpts in Appendix C).

Scenario	Injected fault	Observed invariant	Evidence
----------	----------------	--------------------	----------

1. Replica crash mid-request	Replica crash-C is started with --fault crash-after-recv: on its first Increment it logs the receipt and terminates its own process without replying.	The client commits with 2 of 3 acks and no exception reaches the caller; the crashed process exits with code 1; both surviving replicas hold x = 1.	test_replica_crash_mid_request_still_commits (passed). Log of crash-C: "[crash-C] RECV Increment(counter=x, delta=1) L=4 (received L=3)", and no APPLY or SEND line follows.
2. Request duplication	The same idempotency key (dup-1) is sent twice through the quorum client and once more directly to each replica.	The value stays 5, not 10; every direct resend returns was_duplicate = true.	test_duplicate_request_moves_value_once (passed). Log of dup-A: the first request gives "RECV L=2, APPLY counter=x -> 5 L=3"; the repeated requests give "RECV L=10, DEDUP counter=x stays 5 L=11" and "RECV L=13, DEDUP counter=x stays 5 L=14".
3. Induced timeout with retry	Replica timeout-A is started with --fault slow-first --delay-ms 1500 (the first request is delayed 1.5 s); the client deadline is 0.5 s.	The first attempt hits DEADLINE_EXCEEDED; the retry with the same key succeeds (new_value = 1, was_duplicate = false); the delayed original is later recognised as a duplicate; the counter moved exactly once (Get = 1).	test_induced_timeout_retry_moves_exactly_once (passed). Log of timeout-A: "RECV L=2" (delayed original), "RECV L=3" (retry), "APPLY counter=x -> 1 L=4", "SEND L=5", "DEDUP counter=x stays 1 L=6" (original completes).

In scenarios 2 and 3 the log contains exactly one APPLY per idempotency key; every further request with that key produces a DEDUP event instead. Majority and below-majority behaviour are verified separately by test_majority_commit_two_acks (TC-09) and test_no_commit_below_majority (TC-10).

Task 5 Performance

Measured with tests/perf_benchmark.py (raw output: notes/c4_perf_results.md). Method: replicas run as separate processes on the same machine; clients are threads of one Python process. Each client sends 2,000 / (number of clients) increments with a fresh idempotency key after a 50-request warm-up that is excluded. Latency is measured with time.perf_counter around each client call. p95 is the value at rank ceil(0.95 * n) of the sorted latencies. Console output of the replicas was disabled (--quiet).

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.41	0.80	2000
Single replica, 16 clients	3.41	4.18	2000
Quorum (3 replicas), 1 client	0.83	1.41	2000
Quorum (3 replicas), 16 clients	8.48	13.32	2000

With one client the quorum path is about 2 times slower than a single replica (median 0.83 ms vs 0.41 ms), and with 16 clients the gap widens to about 2.5 times in the median (8.48 ms vs 3.41 ms) and about 3.2 times at p95 (13.32 ms vs 4.18 ms). The extra latency has three sources: every increment is fanned out to three replicas through a thread pool, the client waits for all three replies and therefore for the slowest one, and the three replicas do three times the work on the same machine and compete for the same CPU cores. This competition for the CPU, together with the Python GIL in the client, explains why the gap grows with 16 clients and why at 16 clients the p95 gap (3.2 times) is larger than the median gap (2.5 times). On separate machines the three calls would run in parallel, so the cost would approach one network round trip to the slowest replica plus the fan-out overhead, not three sequential round trips. For a like counter this is acceptable: a few milliseconds of extra latency per increment is invisible to users, and it buys tolerance of one replica crash without losing a committed write.

Task 6

Known limitations

1. No persistence. Each replica keeps its counters and its deduplication store only in memory. After a restart a replica starts empty, so committed writes survive only as long as a majority of replicas stays up. Persisting an append-only log to disk would remove this limitation.
2. A write waits for all three replicas. The quorum client commits with 2 acknowledgements but still waits for the third reply. If one replica is down, every write is delayed by the retry back-off to that replica ($0.2 + 0.4 + 0.8 \text{ s} = 1.4 \text{ s}$ at least), although two acknowledgements are already available. Returning as soon as the majority has answered would remove this delay.
3. No consensus, no repair, no rollback. A write that reached only one replica is not undone: `test_no_commit_below_majority` shows that the client receives `QuorumError` while the surviving replica holds `x = 1`. A replica that was down also never catches up on the writes it missed, and reads come from a single replica and may be stale; `test_replicas_converge` covers only the replicas that stayed alive, and there is no automated test for stale reads.
4. Unbounded deduplication store. Every idempotency key is kept forever, so the store grows with the number of operations. A real service would expire keys after the maximum retry window.
5. Indicative performance results. Clients and replicas ran on one 8 GB laptop and shared the CPU, and 16 client threads share the Python GIL, so the absolute numbers in Section 5 would differ on separate machines. Each configuration was measured in a single run.

Task 7

Design notes and ordering analysis

7.1 RPC semantics and retries (A2)	The service achieves at-most-once execution of the mutation. After a timeout the client cannot tell whether the request was lost or only the reply was lost, so it retries. Every logical operation carries a client-generated idempotency key, which stays the same across all retry attempts. The server checks this key in a deduplication store under the same lock that mutates the counter; a repeated key returns the stored result with <code>was_duplicate = true</code> and the delta is not applied again. Therefore a retry is safe: the counter changes at most once per logical operation.
------------------------------------	--

7.2 Concurrency (A3)	The server handles clients in parallel with <code>grpc.server(futures.ThreadPoolExecutor(max_workers=8))</code> . All shared state (the counter values and the deduplication store) is protected by a single <code>threading.Lock</code> . The duplicate-key check and the counter update happen inside the same critical section, so two concurrent requests cannot both pass the check. This is verified by <code>test_concurrent_increments_exact</code> : two concurrent clients x 1,000 increments yield exactly 2,000.
7.3 Lamport clocks (B1)	Each process (client, replica) keeps its own logical counter in <code>clocks.py</code> . Before every local event (SEND, APPLY) the counter is incremented; on receiving a message it is set to <code>max(own, received) + 1</code> before processing. The current value is attached to every outgoing message in the <code>lamport_time</code> field. Every event is written to the log in the required format, and the counter update and the log write happen under one lock, so the order of lines in a log matches the order of L values.
7.4 Ordering analysis of the trace (B2)	(1) Causally ordered pairs. First, client-2 SEND Increment(y) (L=1) happened before client-1 RECV IncrementReply(x=1) (L=8). Chain: client-2 SEND (L=1) -> replica-A RECV (L=2) by message; replica-A RECV (L=2) -> APPLY y=1 (L=3) -> SEND (L=4) -> RECV (L=5) -> APPLY x=1 (L=6) -> SEND (L=7) by local order on the replica; replica-A SEND (L=7) -> client-1 RECV (L=8) by message. The clients never talk to each other, but the replica handled client-2's request before replying to client-1, so the order is causal. Second, replica-A APPLY x=2 (L=12) happened before client-3 RECV GetReply(value=2) (L=16). Chain: replica-A APPLY (L=12) -> SEND (L=13) -> RECV Get (L=14) -> SEND GetReply (L=15) by local order; replica-A SEND (L=15) -> client-3 RECV (L=16) by message. This is why client-3 reads x=2. (2) Concurrent pair. client-1 SEND Increment(x) (L=1) and client-2 SEND Increment(y) (L=6) are concurrent. The client-1 SEND is the first event of client-1, so nothing precedes it and client-2 SEND (L=6) cannot happen before it. In the other direction, the only path from client-1 goes through replica-A RECV (L=5), but the last message client-2 received before its SEND (L=6) was the reply sent at replica-A L=4, before that RECV, so it carries no information about client-1's request. The values $L=1 < L=6$ only reflect how the counters happened to advance: Lamport clocks guarantee that $a \rightarrow b$ implies $L(a) < L(b)$, not the converse, so a smaller L does not mean the event happened earlier, and the values cannot show that two events are concurrent. (3) Limitation. Lamport clocks cannot distinguish causal order from concurrency: $L(a) < L(b)$ does not imply $a \rightarrow b$. Vector clocks fix this, since two events are concurrent exactly when neither vector is componentwise less than or equal to the other.
7.5 Replication and quorum (C1)	If the client reports a failed write because only one replica acknowledged it, that replica has nevertheless applied the increment, so a later read from it returns the new value although the client was told the operation failed (a partially applied write). A consensus protocol such as Raft or Paxos prevents this: a value becomes visible only after a majority has accepted it in an agreed order, and an entry that reached only a minority is never served as committed. Get reads from a single replica, so it can return a stale value if that replica missed a write (for example because it was down or slow). Reads are therefore not guaranteed to reflect the latest committed write.

7.4

The trace below was produced by `scenario_b2.py`: client-1 increments x twice, client-2 increments y twice, and client-3 only reads x and y. The merged log of the four processes (28 lines) is in Appendix B; in the text, an event is identified by its process and its Lamport value, for example "client-2 L=1".

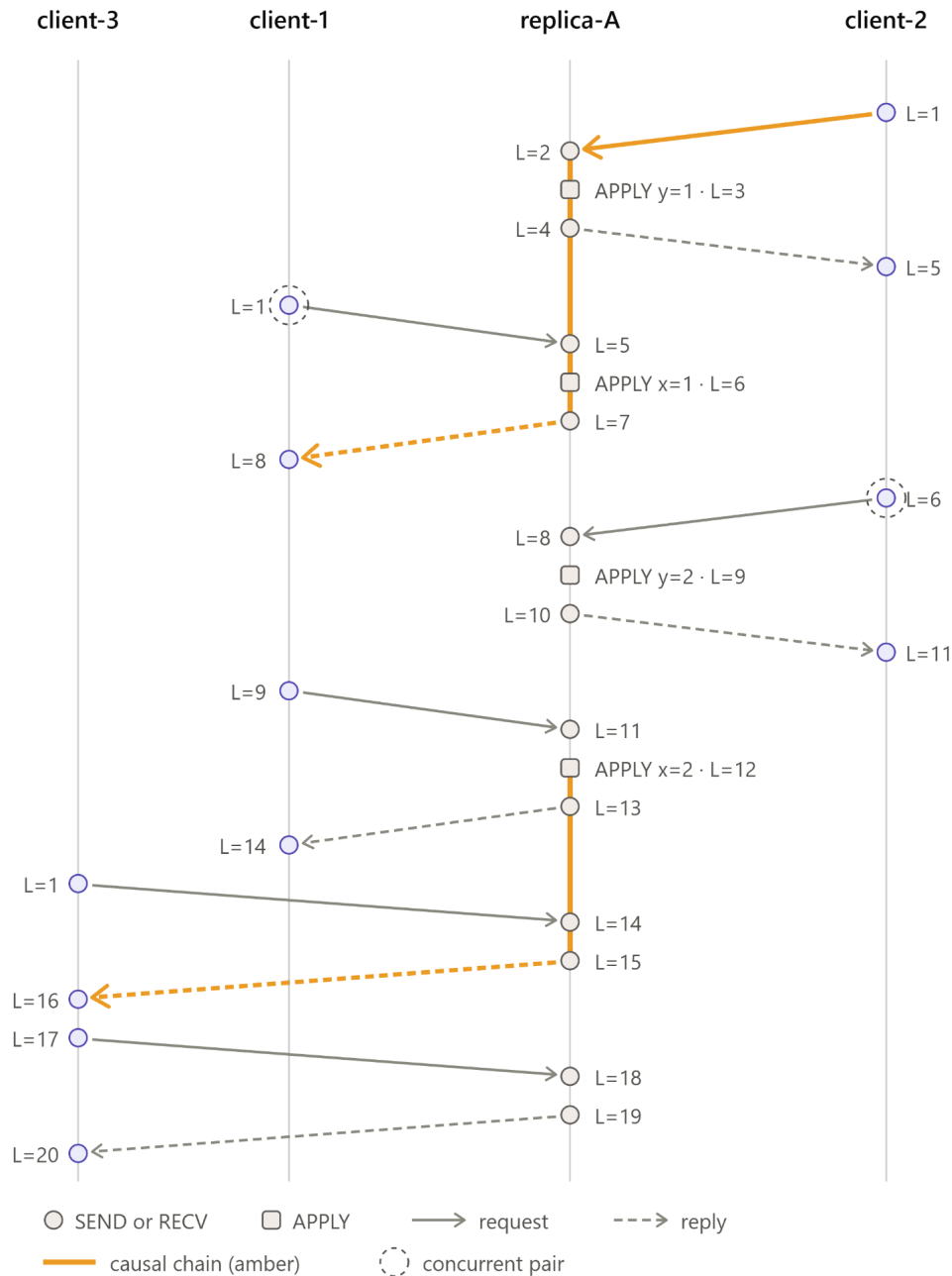


Figure 1. Happened-before diagram of the trace in Appendix B (circles: send and receive events, squares: apply events, amber: the two causal chains, dashed rings: the concurrent pair). The vertical position is only for readability.

Task 8

Reflection questions

Q1. After a timeout, why is it impossible for the client to distinguish request lost from reply lost, and how does your idempotency design make that ambiguity harmless?

After a timeout the client sees only the absence of a reply. Three different situations produce exactly this observation: the request was lost before it reached the replica, the replica crashed or was too slow before applying it, or the request was applied and only the reply was lost. The network gives no bound on delay, so the client cannot wait long enough to be sure, and the status codes (DEADLINE_EXCEEDED, UNAVAILABLE) are the same in all three cases. The design makes the ambiguity harmless: the idempotency key is generated once per logical operation and reused by every retry, and the replica stores the key together with its result under the same lock that updates the counter. Resending is therefore safe in every case. If the original was lost, the retry applies the delta; if it was already applied,

the retry returns the stored result with `was_duplicate = true` and the counter does not change. This is verified by `test_retry_after_timeout_is_safe` (TC-08) and `test_induced_timeout_retry_moves_exactly_once` (TC-14), where the log of timeout-A shows one APPLY and one DEDUP for the same operation.

Q2. Your Task B2 trace contains a pair of concurrent events whose Lamport values differ. Explain why the smaller value does not imply that the event happened first.

The pair is client-1 SEND Increment(x) with $L=1$ and client-2 SEND Increment(y) with $L=6$ (Section 7.4). A Lamport clock guarantees only one direction: if a happened before b, then $L(a) < L(b)$. The converse is false. The counter of a process grows with the events that causally precede it, and the counters of two processes are related only through messages. Client-2 reached $L=6$ simply because it had already completed a full round trip (events $L=1$ to $L=5$) before its second send, while client-1 had done nothing yet and was still at $L=1$. No chain of local steps and messages connects the two sends in either direction, so the trace does not determine which of them occurred first in real time; the values compare how much each process had done, not when. Telling such pairs apart from causally ordered ones requires vector clocks.

Q3. With three replicas and majority commit, which failures can the service tolerate while preserving every committed write, and which failure breaks the guarantee?

Relate your answer to the safety/liveness distinction from Week 5. A committed write has been acknowledged by at least two of the three replicas, so it exists on at least one of any two surviving replicas. The service therefore tolerates the loss of any single replica (a crash, or a replica that is slow or unreachable) without losing a committed write: `test_majority_commit_two_acks` (TC-09) and `test_replica_crash_mid_request_still_commits` (TC-12) show that writes keep committing with two replicas. Duplicated or retried requests are tolerated through idempotency (TC-05, TC-13). The guarantee breaks when two replicas are lost: with only one replica left no write can commit, and if the two lost replicas were exactly those that acknowledged an earlier write, that write is gone. In terms of Week 5, safety means that nothing bad happens (a committed write is never lost and is not applied twice), and liveness means that something good eventually happens (a write eventually commits). With one failure both hold. With two failures liveness is lost on purpose: the client reports QuorumError instead of pretending success (TC-10), which is the acceptable direction, because the system stops rather than lie. Safety is endangered only when two replicas are lost together; in our memory-only implementation a restart of two replicas has the same effect (Section 6, limitation 1). Single-replica reads can also return a stale value, and a write reported as failed can remain visible on one replica (TC-10), so the guarantee covers committed writes only.

Q4. If this service had to survive replica restarts without losing committed writes, what is the smallest change you would make, and which Week 6 mechanism does it anticipate?

The smallest change is to make each replica durable: before replying to an Increment, append a record (counter id, delta, idempotency key, resulting value) to an append-only log file on disk and flush it, and on startup replay this log to rebuild both the counter values and the deduplication store. Rebuilding the deduplication store matters, because retries that arrive after a restart must still be recognised as duplicates. This anticipates the persistent log of Raft (Week 6): a replica must write an entry to stable storage before it acknowledges it, so that an acknowledged entry survives a crash. It is only the first step: a restarted replica would still miss the writes that happened while it was down, which Raft solves with log replication and catch-up from the leader.

Appendices

Appendix A. Output of `pip freeze` in the virtual environment used for all runs.

```
colorama==0.4.6
grpcio==1.84.0
grpcio-tools==1.84.0
iniconfig==2.3.1
packaging==26.3
pluggy==1.6.0
protobuf==7.36.2
Pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0
```

Appendix B. Merged event logs of client-1, client-2, client-3 and replica-A from scenario_b2.py (file logs/b2_trace.txt); "received L" is the timestamp carried by the incoming message.

```
[client-1] SEND      Increment(counter=x, delta=1)                L=1
[client-1] RECV      IncrementReply(new_value=1)                L=8   (received L=7)
[client-1] SEND      Increment(counter=x, delta=1)                L=9
[client-1] RECV      IncrementReply(new_value=2)                L=14  (received L=13)
[client-2] SEND      Increment(counter=y, delta=1)                L=1
[client-2] RECV      IncrementReply(new_value=1)                L=5   (received L=4)
[client-2] SEND      Increment(counter=y, delta=1)                L=6
[client-2] RECV      IncrementReply(new_value=2)                L=11  (received L=10)
[client-3] SEND      Get(counter=x)                               L=1
[client-3] RECV      GetReply(value=2, found=True)              L=16  (received L=15)
[client-3] SEND      Get(counter=y)                               L=17
[client-3] RECV      GetReply(value=2, found=True)              L=20  (received L=19)
[replica-A] RECV      Increment(counter=y, delta=1)              L=2   (received L=1)
[replica-A] APPLY     counter=y -> 1                             L=3
[replica-A] SEND      IncrementReply(new_value=1)                L=4
[replica-A] RECV      Increment(counter=x, delta=1)              L=5   (received L=1)
[replica-A] APPLY     counter=x -> 1                             L=6
[replica-A] SEND      IncrementReply(new_value=1)                L=7
[replica-A] RECV      Increment(counter=y, delta=1)              L=8   (received L=6)
[replica-A] APPLY     counter=y -> 2                             L=9
[replica-A] SEND      IncrementReply(new_value=2)                L=10
[replica-A] RECV      Increment(counter=x, delta=1)              L=11  (received L=9)
[replica-A] APPLY     counter=x -> 2                             L=12
[replica-A] SEND      IncrementReply(new_value=2)                L=13
[replica-A] RECV      Get(counter=x)                             L=14  (received L=1)
[replica-A] SEND      GetReply(value=2, found=True)              L=15
[replica-A] RECV      Get(counter=y)                             L=18  (received L=17)
[replica-A] SEND      GetReply(value=2, found=True)              L=19
```

Appendix C. Excerpts from logs/failures/: crash-C (scenario 1), dup-A (scenario 2), timeout-A (scenario 3).

```
[crash-C] RECV      Increment(counter=x, delta=1)                L=4   (received L=3)
[dup-A] RECV      Increment(counter=x, delta=5)                  L=2   (received L=1)
[dup-A] APPLY     counter=x -> 5                                 L=3
[dup-A] SEND      IncrementReply(new_value=5)                    L=4
[dup-A] RECV      Increment(counter=x, delta=5)                  L=10  (received L=9)
[dup-A] DEDUP     counter=x stays 5                             L=11
[dup-A] SEND      IncrementReply(new_value=5)                    L=12
[dup-A] RECV      Increment(counter=x, delta=5)                  L=13  (received L=1)
[dup-A] DEDUP     counter=x stays 5                             L=14
[dup-A] SEND      IncrementReply(new_value=5)                    L=15
[dup-A] RECV      Get(counter=x)                                L=18  (received L=17)
[dup-A] SEND      GetReply(value=5, found=True)                  L=19
[timeout-A] RECV     Increment(counter=x, delta=1)                L=2   (received L=1)
[timeout-A] RECV     Increment(counter=x, delta=1)                L=3   (received L=2)
[timeout-A] APPLY     counter=x -> 1                             L=4
[timeout-A] SEND      IncrementReply(new_value=1)                L=5
[timeout-A] DEDUP     counter=x stays 1                         L=6
[timeout-A] SEND      IncrementReply(new_value=1)                L=7
[timeout-A] RECV      Get(counter=x)                             L=8   (received L=7)
[timeout-A] SEND      GetReply(value=1, found=True)              L=9
```

Appendix D. Output of `python -m pytest -v`: 14 passed in 17.59 s.

```
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.1.1, pluggy-1.6.0 --
C:\Users\Dauren1\Desktop\dcc-assignment2\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Dauren1\Desktop\dcc-assignment2
configfile: pytest.ini
collecting ... collected 14 items

tests/test_clocks.py::test_local_event_increments_clock PASSED [ 7%]
tests/test_clocks.py::test_receive_takes_max_plus_one PASSED [ 14%]
tests/test_clocks.py::test_receive_with_smaller_value_still_increments PASSED [ 21%]
tests/test_counter.py::test_increment_applies_delta PASSED [ 28%]
tests/test_counter.py::test_duplicate_key_not_reapplied PASSED [ 35%]
tests/test_counter.py::test_get_missing_counter PASSED [ 42%]
tests/test_counter.py::test_concurrent_increments_exact PASSED [ 50%]
tests/test_counter.py::test_retry_after_timeout_is_safe PASSED [ 57%]
tests/test_counter.py::test_majority_commit_two_acks PASSED [ 64%]
tests/test_counter.py::test_no_commit_below_majority PASSED [ 71%]
tests/test_counter.py::test_replicas_converge PASSED [ 78%]
tests/test_failures.py::test_replica_crash_mid_request_still_commits PASSED [ 85%]
tests/test_failures.py::test_duplicate_request_moves_value_once PASSED [ 92%]
tests/test_failures.py::test_induced_timeout_retry_moves_exactly_once PASSED [100%]

===== 14 passed in 17.59s =====
```